Pertemuan 08 — Ujian Tengah Semester (UTS)

Versi PDF dari modul daring. Versi terbaru, kode praktikum, dan tautan aktif:
https://github.com/antonprafanto/embeddedsystem/tree/main/pertemuan-08-uts

> Saatnya mengukur seberapa jauh kamu sudah melangkah. Halaman ini memuat informasi ujian, kisi-kisi, lembar ringkas API dan rumus, contoh soal beserta pembahasannya, serta satu latihan proyek mini untuk simulasi ujian praktik.

⏱️ **Durasi**	Teori ±90 menit + praktik ±120 menit *(sesuaikan)*
📋 **Cakupan**	Pertemuan 01–07 (CPMK-1, CPMK-2, CPMK-3)
🧰 **Alat**	Alat tulis dan kalkulator; laptop + Wokwi; *board* ESP32 + kit bila ujian praktik memakai *board*
📥 **Slide**	👁️ Lihat di peramban · ⬇️ Unduh PDF · ⬇️ Unduh PPTX
📄 **Modul PDF**	⬇️ Unduh modul ini dalam PDF — untuk dibaca luring atau dicetak

> **Catatan / Note**
>
> **Untuk dosen:** semua bagian bertanda *(sesuaikan)*, yaitu waktu, durasi, sifat ujian, komposisi soal, dan bobot, adalah usulan. Silakan ubah sesuai ketentuan program studi sebelum halaman ini dibagikan.

Daftar Isi

- 🎯 Apa yang Diukur di UTS?
- 📅 Informasi Ujian
- 🧭 Cakupan dan Kisi-Kisi
- 📝 Format Ujian
- 📋 Lembar Ringkas API
- 🧮 Rumus Penting
- 🧠 Contoh Soal Teori
- 🔧 Contoh Soal Praktik dan Rubrik
- 🚦 Latihan Proyek Mini: Lampu Penyeberangan Pintar
- 💡 Tips Belajar
- 📏 Tata Tertib Ujian
- 🚫 Kesalahan Umum
- 📚 Referensi
- 🖼️ Sumber Gambar

🎯 Apa yang Diukur di UTS?

UTS bukan lomba menghafal. Yang diukur adalah kemampuanmu **memahami, menghitung, dan membangun** sistem kecil dengan ESP32. Setelah belajar dengan halaman ini, kamu diharapkan bisa:

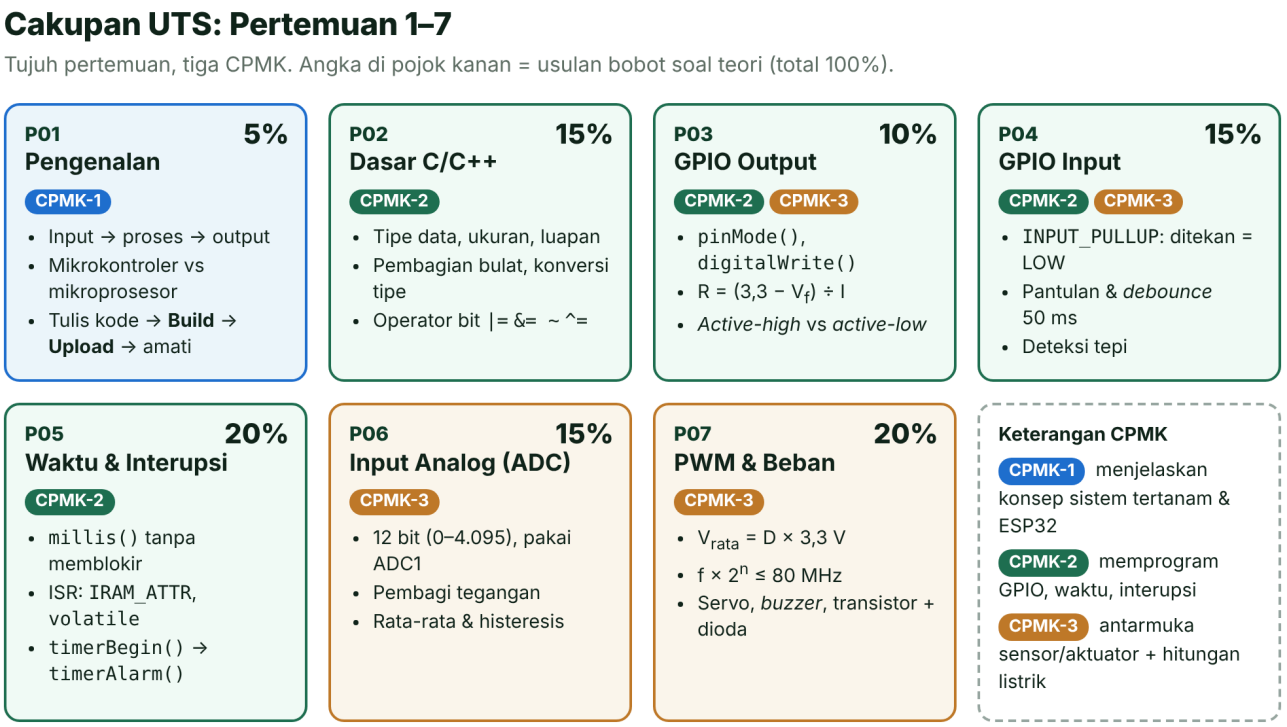
1. Menjelaskan konsep sistem tertanam, arsitektur ESP32, dan alur kerja *firmware* (CPMK-1).
2. Menulis program C/C++ yang mengendalikan GPIO, pewaktuan `millis()`, interupsi, dan *timer* tanpa memblokir (CPMK-2).
3. Merangkai dan menghitung antarmuka sederhana: resistor LED, *pull-up*, pembagi tegangan, ADC, PWM, servo, *buzzer*, dan transistor (CPMK-3).
4. Menggabungkan beberapa topik itu dalam satu program yang rapi dalam waktu terbatas, lalu menjelaskannya secara lisan.

📅 Informasi Ujian

Butir	Ketentuan
Hari, tanggal	Pertemuan 8 — *(sesuaikan)*
Waktu	Teori: *(sesuaikan)* · Praktik: *(sesuaikan)*
Durasi	Teori ±90 menit · Praktik ±120 menit *(sesuaikan)*
Tempat	*(sesuaikan)*
Sifat ujian teori	Buku tertutup. Lembar ringkas API dan rumus disediakan pengawas. Kalkulator biasa boleh, ponsel tidak *(sesuaikan)*.
Sifat ujian praktik	Buku terbuka terbatas: boleh membuka modul P01–P07 dan dokumentasi resmi. **Tanpa** AI, aplikasi pesan, atau berbagi berkas *(sesuaikan)*.
Perangkat praktik	Laptop dengan Wokwi (wajib siap), atau *board* ESP32 + kit bila ditentukan *(sesuaikan)*
Bobot	UTS = 25% nilai akhir (lihat RPS); di dalamnya teori 50% + praktik 50% *(sesuaikan)*

🧭 Cakupan dan Kisi-Kisi

UTS mencakup tujuh pertemuan pertama. Gambar berikut merangkum isi tiap pertemuan beserta CPMK dan usulan bobotnya.



Gambar 8.1 — Peta cakupan UTS: Pertemuan 1–7. Sumber: dibuat untuk materi ini.

No	Topik (modul)	CPMK	Indikator yang diukur	Bentuk soal	Bobot teori
1	Sistem tertanam, ESP32, dan alur kerja (P01)	1	Menjelaskan pola input–proses–output, membedakan mikrokontroler dengan mikroprosesor, mengenali tahap **Build**/**Upload** beserta galatnya	PG, esai	5%
2	Dasar C/C++ (P02)	2	Memilih tipe data, memprediksi luapan dan pembagian bulat, memakai operator bit, membaca alur program	PG, isian	15%
3	GPIO output (P03)	2, 3	Menghitung resistor LED, membedakan *active-high* dan *active-low*, memilih pin yang aman	PG, perhitungan	10%
4	GPIO input (P04)	2, 3	Menjelaskan *pull-up*/*pull-down* dan ambang HIGH/LOW, mengatasi pantulan dengan *debouncing*, deteksi tepi	PG, esai	15%
5	Waktu dan interupsi (P05)	2	Memakai pola `millis()` dan menjelaskan luapan, menulis ISR yang aman, menghitung nilai alarm *timer*	PG, perhitungan, esai	20%
6	Input analog: ADC (P06)	3	Mengonversi nilai ADC, memilih ADC1/ADC2, menghitung pembagi tegangan, menerapkan rata-rata dan histeresis	PG, perhitungan, esai	15%

No	Topik (modul)	CPMK	Indikator yang diukur	Bentuk soal	Bobot teori
7	PWM dan beban (P07)	3	Menghitung siklus kerja dan tegangan rata-rata, batas frekuensi–resolusi LEDC, pulsa servo, nada, resistor basis transistor	PG, perhitungan	20%
				Jumlah	**100%**

Ujian praktik mengukur CPMK-2 dan CPMK-3 sekaligus: rangkaian yang benar dan aman, program yang tidak memblokir, serta perpaduan minimal tiga topik (misalnya GPIO, tombol dengan interupsi, `millis()`, ADC, dan PWM).

> **Tips / Tip**
>
> **Cara memakai kisi-kisi:** untuk setiap baris, buka bagian "Tujuan Pembelajaran" dan "Cek Pemahaman" di modul yang ditautkan. Lalu uji dirimu: bisakah kamu mengerjakan tiap indikator tanpa melihat catatan?

📝 Format Ujian



Gambar 8.2 — Usulan format UTS. Sumber: dibuat untuk materi ini (ikon Font Awesome Free, CC BY 4.0).

Ujian Teori

Bagian	Jumlah soal	Poin	Isi
A. Pilihan ganda	20	20 × 2 = 40	Konsep, membaca potongan kode, memilih pin dan fungsi

Bagian	Jumlah soal	Poin	Isi
B. Isian dan perhitungan	5	5 × 8 = 40	Resistor, ADC, pembagi tegangan, PWM, `millis()`, *timer*. Tulis rumus, substitusi, hasil, dan satuan.
C. Esai singkat	2	2 × 10 = 20	Menjelaskan "mengapa", mencari dan memperbaiki kesalahan kode

Ujian Praktik

Kamu menerima satu soal yang memadukan beberapa topik, mirip latihan proyek mini di bawah. Kerjakan di Wokwi atau di *board* sesuai ketentuan dosen. Di akhir waktu, kumpulkan *folder* proyek dalam satu berkas ZIP, misalnya `UTS_NIM_Nama.zip` *(sesuaikan)*, berisi `src/main.cpp`, `platformio.ini`, dan `diagram.json` atau foto rangkaian. Setelah itu ada demo singkat: tunjukkan fiturnya, jawab satu–dua pertanyaan, lalu lakukan satu perubahan kecil yang diminta penguji (misalnya mengganti durasi atau pin).

> **Penting / Important**
>
> **Nilai parsial itu nyata.** Program yang belum lengkap tetapi berjalan dan rapi tetap mendapat nilai. Simpan salinan versi yang sudah berjalan sebelum menambah fitur baru, supaya kamu selalu punya sesuatu untuk dikumpulkan.

📋 Lembar Ringkas API

Semua fungsi di bawah berlaku untuk Arduino-ESP32 3.x (versi 3.3 yang kita pakai). API lama core 2.x seperti `ledcSetup()` dan `ledcAttachPin()` **sudah tidak ada** dan akan menghasilkan galat kompilasi.

GPIO, Waktu, dan Interupsi

Fungsi	Kegunaan	Contoh
`pinMode(pin, mode)`	Mengatur pin: `OUTPUT`, `INPUT`, `INPUT_PULLUP`, `INPUT_PULLDOWN`	`pinMode(4, INPUT_PULLUP);`
`digitalWrite(pin, nilai)`	Menulis `HIGH` (≈ 3,3 V) atau `LOW` (0 V)	`digitalWrite(23, HIGH);`
`digitalRead(pin)`	Membaca `HIGH` / `LOW`. Dengan *pull-up*, tombol ditekan = `LOW`	`if (digitalRead(4) == LOW) { … }`

Fungsi	Kegunaan	Contoh
`millis()`	Milidetik sejak menyala, `unsigned long` 32 bit, meluap setelah ±49,7 hari	`unsigned long t = millis();`
`micros()`	Mikrodetik sejak menyala, meluap setelah ±71,6 menit	`unsigned long t0 = micros();`
`attachInterrupt(digitalPinToInterrupt(pin), isr, mode)`	Memanggil ISR saat tepi `RISING`, `FALLING`, atau `CHANGE`	`attachInterrupt(digitalPinToInterru isrTombol, FALLING);`
`portENTER_CRITICAL(&kunci)` dan `portEXIT_CRITICAL(&kunci)`	Bagian kritis di `loop()`; di dalam ISR pakai versi berakhiran `_ISR`	`portMUX_TYPE kunci = portMUX_INITIALIZER_UNLOCKED;`
`timerBegin(frekHz)`	Membuat *timer* dengan frekuensi hitung tertentu	`hw_timer_t *t = timerBegin(1000000)` = 1 µs)
`timerAttachInterrupt(t, isr)`	Memasang ISR *timer*	`timerAttachInterrupt(t, &isrTimer);`
`timerAlarm(t, nilai, ulang, jumlah)`	Alarm setiap `nilai` tik; `ulang = true` berarti berulang; `jumlah = 0` tanpa batas	`timerAlarm(t, 500000, true, 0);` (tia s)

ADC, PWM, dan Serial

Fungsi	Kegunaan	Contoh
`analogRead(pin)`	Nilai mentah 0–4.095 (12 bit)	`int n = analogRead(34);`
`analogReadMilliVolts(pin)`	Tegangan dalam mV, terkalibrasi eFuse	`uint32_t mv = analogReadMilliVolts(34);`
`analogReadResolution(bit)` dan `analogSetAttenuation(ADC_11db)`	Resolusi dan atenuasi (bawaan: 12 bit, 11 dB)	`analogReadResolution(12);`

Fungsi	Kegunaan	Contoh
`ledcAttach(pin, frek, bit)`	Memasang pin ke PWM LEDC; mengembalikan `false` bila gagal	`ledcAttach(23, 5000, 8);`
`ledcWrite(pin, duty)`	Siklus kerja 0 sampai $2^n - 1$	`ledcWrite(23, 128);` (±50% pada 8 bit)
`ledcWriteTone(pin, frek)`	Nada dengan siklus kerja 50%; frekuensi 0 = diam	`ledcWriteTone(18, 440);`
`Serial.begin(115200)`	Membuka Serial; samakan dengan `monitor_speed`	`Serial.begin(115200);`
`Serial.print()`, `println()`, `printf()`	Mencetak teks dan angka; format `%d`, `%lu`, `%.2f`, `%s`	`Serial.printf("%lu ms\n", millis());`
`Serial.readStringUntil('\n')` + `trim()`	Membaca satu baris dari Serial Monitor	`String s = Serial.readStringUntil('\n'); s.trim();`

Kerangka Program yang Perlu Kamu Kuasai

```
// 1) Tugas berkala TANPA delay(): pola selisih millis()
const unsigned long INTERVAL_MS = 500;
unsigned long sebelumnya = 0;

void loop() {
  unsigned long sekarang = millis();
  if (sekarang - sebelumnya >= INTERVAL_MS) {   // tetap benar walau millis() meluap
    sebelumnya = sekarang;
    // ... kerjakan tugas berkala di sini ...
  }
  // tugas lain tetap berjalan karena tidak ada yang menunggu
}

// 2) ISR tombol yang aman (pin 4, INPUT_PULLUP, tombol ke GND)
volatile bool ditekan = false;
void IRAM_ATTR isrTombol() { ditekan = true; }   // singkat: tanpa Serial, tanpa delay()
// di setup(): attachInterrupt(digitalPinToInterrupt(4), isrTombol, FALLING);

// 3) Timer perangkat keras: ISR setiap 0,5 detik
hw_timer_t *timerKu = nullptr;
volatile bool tik = false;
void IRAM_ATTR isrTimer() { tik = true; }
// di setup(): timerKu = timerBegin(1000000);          // 1 MHz → 1 tik = 1 µs
//             timerAttachInterrupt(timerKu, &isrTimer);
//             timerAlarm(timerKu, 500000, true, 0);     // 500.000 tik = 0,5 s

// 4) PWM dan nada
// ledcAttach(23, 5000, 8);   ledcWrite(23, 128);        // LED ±50%
// ledcAttach(18, 1000, 10);  ledcWriteTone(18, 440);    // nada A4; 0 = diam
```

🧮 Rumus Penting

Semua contoh di tabel ini sudah dihitung ulang. Biasakan menulis **rumus → substitusi → hasil + satuan**, karena langkah yang benar tetap dinilai walaupun angka akhirmu meleset.

Besaran	Rumus	Contoh
Resistor LED	$R = (3{,}3\ V - V_f) \div I$	(3,3 − 2,0) ÷ 0,010 = 130 Ω → pilih 220 Ω (I ≈ 5,9 mA)
Daya resistor	$P = I^2 \times R$	$(0{,}0059)^2 \times 220 \approx 7{,}7$ mW, aman untuk resistor ¼ W
Arus *pull-up* saat tombol ditekan	$I = 3{,}3\ V \div R$	10 kΩ → 0,33 mA; internal ±45 kΩ → ±73 µA
Ambang logika (VDD = 3,3 V)	HIGH ≥ 0,75 × VDD; LOW ≤ 0,25 × VDD	HIGH ≥ 2,475 V; LOW ≤ 0,825 V
Pembagi tegangan	$V_{out} = V_{in} \times R2 \div (R1 + R2)$	3,3 × 10 ÷ (10 + 10) = 1,65 V
Konversi ADC 12 bit	V ≈ nilai × 3,3 ÷ 4.095; langkah ≈ 0,806 mV	2.048 → ≈ 1,650 V
Siklus kerja dan tegangan rata-rata	$D = \text{duty} \div (2^n - 1)$; $V_{rata} = D \times 3{,}3\ V$	8 bit, duty 64 → D ≈ 25,1% → ≈ 0,83 V
Batas frekuensi–resolusi LEDC	$f \times 2^n \le 80\ MHz \rightarrow n_{maks} = \lfloor \log_2(80\ MHz \div f) \rfloor$	1 kHz → 16 bit; 5 kHz → 13 bit; 20 kHz → 11 bit
Lebar pulsa servo (rentang praktikum P07)	$t = 500 + (\text{sudut} \div 180) \times 2.000\ \mu s$, periode 20 ms	90° → 1.500 µs (7,5% dari 20 ms)
Frekuensi nada	$f = 440 \times 2^{n/12}$, n = jarak semiton dari A4	C5 (n = 3) → ≈ 523,3 Hz
Resistor basis transistor	$I_B = (I_C \div h_{FE,min}) \times (2\text{–}3)$; $R_B = (3{,}3 - 0{,}7) \div I_B$	Lihat Soal 13
Debounce	Terima keadaan baru bila sudah stabil ≥ 50 ms	Tombol memantul dalam orde milidetik
Selisih waktu	`sekarang - sebelumnya >= INTERVAL` dengan `unsigned long`	Tetap benar saat `millis()` meluap (tiap 2^{32} ms ≈ 49,7 hari)
Nilai alarm *timer*	alarm = frekuensi *timer* × selang	1 MHz × 0,5 s = 500.000 tik

🧠 Contoh Soal Teori

Ada 16 contoh soal: 6 pilihan ganda, 7 isian/perhitungan, dan 3 esai singkat. Kerjakan dulu tanpa membuka pembahasan, baru klik **Lihat pembahasan** untuk mencocokkan.

A. Pilihan Ganda

Soal 1 (P01 · CPMK-1). Saat kamu menekan **Build**, PlatformIO menampilkan galat `expected ';' before '}' token`. Pernyataan yang benar adalah ...

- A. Galat terjadi saat *upload*, sehingga kabel USB perlu diganti
- B. Galat terjadi saat kompilasi, sehingga `firmware.bin` belum terbentuk dan belum ada yang bisa diunggah
- C. Galat terjadi saat program berjalan di ESP32
- D. Galat terjadi karena *baud rate* Serial Monitor berbeda

▶ Lihat pembahasan

Soal 2 (P02 · CPMK-2). Berapa nilai `x` setelah kode berikut dijalankan?

```
uint8_t x = 250;
x = x + 10;
```

- A. 260
- B. 4
- C. 255
- D. Galat kompilasi

▶ Lihat pembahasan

Soal 3 (P03 · CPMK-2, 3). Anoda LED disambung ke 3V3, sedangkan katodenya disambung lewat resistor 220 Ω ke GPIO 23. Perintah untuk **menyalakan** LED adalah ...

- A. `digitalWrite(23, HIGH);`
- B. `digitalWrite(23, LOW);`
- C. `pinMode(23, INPUT);`
- D. `analogRead(23);`

▶ Lihat pembahasan

Soal 4 (P04 · CPMK-2, 3). Sebuah tombol dipasang antara GPIO 34 dan GND dengan `pinMode(34, INPUT_PULLUP);`. Saat tombol **tidak** ditekan, bacaannya acak. Penyebabnya adalah ...

- A. GPIO 34 hanya bisa dipakai sebagai output
- B. GPIO 34–39 tidak punya resistor *pull-up* internal, sehingga pin mengambang
- C. Tombol seharusnya disambung ke 5 V
- D. `loop()` kurang `delay(50)`

▶ Lihat pembahasan

Soal 5 (P06 · CPMK-3). Sebuah proyek memakai Wi-Fi dan membaca potensiometer. Pin mana yang paling tepat untuk potensiometer?

- A. GPIO 4
- B. GPIO 25
- C. GPIO 35
- D. GPIO 13

▶ Lihat pembahasan

Soal 6 (P07 · CPMK-3). Dengan batas $f \times 2^n \le 80$ MHz, pemanggilan mana yang **tidak mungkin** berhasil?

- A. `ledcAttach(23, 5000, 12);`

- B. `ledcAttach(23, 1000, 16);`
- C. `ledcAttach(23, 40000, 12);`
- D. `ledcAttach(23, 20000, 10);`

▶ Lihat pembahasan

B. Isian dan Perhitungan

Soal 7 (P02 · CPMK-2). Tuliskan nilai akhir `reg` dalam biner, desimal, dan heksadesimal.

```
uint8_t reg = 0b00010110;
reg |= (1 << 0);
reg &= ~(1 << 4);
reg ^= (1 << 7);
```

▶ Lihat pembahasan

Soal 8 (P03 · CPMK-3). LED hijau dengan V_f = 2,1 V akan dinyalakan dari pin 3,3 V dengan target arus 6 mA. Hitung (a) R minimum, (b) nilai standar E12 yang dipilih (sama atau lebih besar), (c) arus sebenarnya, dan (d) daya resistor.

▶ Lihat pembahasan

Soal 9 (P05 · CPMK-2). Perhatikan potongan berikut. Variabel `sebelumnya` dicatat sesaat sebelum `millis()` meluap, dan `sekarang` dibaca sesudahnya.

```
unsigned long sebelumnya = 4294967000UL;
unsigned long sekarang   = 200;
if (sekarang - sebelumnya >= 500) { /* tugas berkala */ }
```

(a) Berapa hasil `sekarang - sebelumnya`? (b) Apakah blok `if` dijalankan? (c) Berapa milidetik lagi blok itu dijalankan?

▶ Lihat pembahasan

Soal 10 (P05 · CPMK-2). *Timer* dibuat dengan `timerBegin(1000000)`. (a) Berapa nilai alarm agar ISR terpanggil setiap 250 ms? (b) Berapa kali per detik ISR terpanggil? (c) Berapa nilai alarmnya jika *timer* dibuat dengan `timerBegin(10000)`?

▶ Lihat pembahasan

Soal 11 (P06 · CPMK-3). (a) `analogRead()` menghasilkan 1.241. Berapa tegangannya (rumus 3,3 ÷ 4.095)? (b) Tegangan baterai dua sel Li-ion (maksimum 8,4 V) diukur lewat pembagi R1 = 27 kΩ (atas) dan R2 = 10 kΩ (bawah, ke GND). Berapa tegangan maksimum di pin, dan apakah aman? (c) Jika `analogReadMilliVolts()` membaca 1.850 mV, berapa tegangan baterainya?

▶ Lihat pembahasan

Soal 12 (P07 · CPMK-3). PWM dipasang dengan `ledcAttach(23, 5000, 10);`. (a) Jika `ledcWrite(23, 768);`, berapa siklus kerja dan tegangan rata-ratanya? (b) Berapa nilai duty agar tegangan rata-ratanya ≈ 1,0 V?

▶ Lihat pembahasan

Soal 13 (P07 · CPMK-3). Kumparan *relay* 5 V berhambatan 125 Ω dikendalikan transistor NPN (anggap h_{FE} minimum = 100, V_{BE} = 0,7 V) dari GPIO 3,3 V dengan *overdrive* 2–3×. Hitung I_C, rentang I_B, rentang R_B, lalu pilih nilai standar. Sebutkan juga komponen pelindung yang wajib dipasang.

▶ Lihat pembahasan

C. Esai Singkat

Soal 14 (P04 · CPMK-2). Seorang mahasiswa membuat *toggle* LED seperti di bawah. Saat tombol ditekan sekali, LED kadang berubah dan kadang tidak. Jelaskan **dua** penyebabnya, lalu tuliskan perbaikannya.

```
void loop() {
  if (digitalRead(PIN_TOMBOL) == LOW) {
    statusLed = !statusLed;
    digitalWrite(PIN_LED, statusLed);
  }
}
```

▶ Lihat pembahasan

Soal 15 (P05 · CPMK-2). Temukan minimal **empat** kesalahan pada kode berikut (tombol dipasang antara GPIO 4 dan GND), lalu perbaiki.

```
int jumlahTekan = 0;

void isrTombol() {
  jumlahTekan++;
  Serial.println("Tombol ditekan!");
  delay(200);   // debounce
}

void setup() {
  Serial.begin(115200);
  pinMode(4, INPUT);
  attachInterrupt(4, isrTombol, RISING);
}
```

▶ Lihat pembahasan

Soal 16 (P06 · CPMK-3). Lampu taman menyala bila nilai sensor cahaya **< 1.000** (gelap) dan padam bila **> 1.200** (terang); di antara keduanya keadaan dipertahankan. Mula-mula lampu padam. Urutan bacaan: 1.300, 1.120, 1.090, 1.110, 1.095, 980, 1.105, 1.210. (a) Tentukan keadaan lampu setelah tiap bacaan. (b) Bandingkan dengan aturan satu ambang "nyala bila < 1.100". (c) Jelaskan mengapa histeresis diperlukan.

▶ Lihat pembahasan

🔧 Contoh Soal Praktik dan Rubrik

Tiga contoh berikut mirip soal praktik UTS. Setiap soal memakai rangkaian dari modul P03–P07, jadi kamu bisa berlatih di Wokwi tanpa komponen tambahan. Kerjakan dengan batas waktu, lalu nilai sendiri memakai rubrik di bawah.

Praktik A — Kedip Dua Kecepatan (±45 menit; P03–P05)

1. LED eksternal di GPIO 23 (lewat 220 Ω) berkedip: nyala 500 ms, padam 500 ms (mode LAMBAT).
2. Setiap tekanan tombol GPIO 4 (`INPUT_PULLUP`) mengganti mode LAMBAT ↔ CEPAT (nyala 100 ms, padam 100 ms).
3. Satu tekanan = satu pergantian: pakai *debounce* 50 ms dan deteksi tepi. Tombol harus tetap tanggap kapan pun ditekan, jadi tanpa `delay()` .
4. Serial mencetak `[INFO] Mode: CEPAT` atau `[INFO] Mode: LAMBAT` setiap kali mode berganti.

▶ Lihat petunjuk

Praktik B — Peredup dengan Peringatan (±60 menit; P06–P07)

1. Potensiometer di GPIO 34 mengatur kecerahan LED GPIO 23 lewat LEDC 5 kHz, 12 bit.
2. Setiap 500 ms, Serial mencetak tegangan potensiometer dalam mV (`analogReadMilliVolts()`) dan siklus kerja dalam persen, misalnya `[DATA] 1650 mV, duty 50%` .
3. Bila siklus kerja > 90%, LED bawaan GPIO 2 berkedip 2 Hz sebagai peringatan "terlalu terang"; selain itu LED bawaan padam.
4. Tanpa `delay()` .

▶ Lihat petunjuk

Praktik C — Lampu Teras Otomatis (±60 menit; P04–P07)

1. Modul LDR (keluaran AO) di GPIO 35. Ukur dulu nilai ADC saat terang dan saat gelap, lalu tetapkan **dua** ambang (histeresis) dan tulis alasannya di komentar.
2. Saat keadaan berubah menjadi gelap, LED GPIO 23 menyala dan *buzzer* pasif GPIO 18 berbunyi 1.000 Hz selama 200 ms tanpa `delay()` . Saat terang, LED padam.
3. Tombol GPIO 4 (interupsi `FALLING` + *debounce*) mengganti mode: OTOMATIS → SELALU NYALA → SELALU PADAM → OTOMATIS.
4. Setiap 1 detik, Serial mencetak nilai ADC, keadaan (gelap/terang), dan mode.

▶ Lihat petunjuk

Rubrik Praktik (Usulan, 100 Poin)

Aspek	Bobot	Penuh	Sebagian	Minim
Rangkaian dan pemilihan pin	20	Benar, rapi, resistor tepat, pin aman, tidak ada tegangan > 3,3 V ke pin	Berfungsi tetapi kurang rapi atau ada pin yang kurang tepat	Salah atau berbahaya (LED tanpa resistor, 5 V ke pin)
Fungsi sesuai spesifikasi	35	Semua butir spesifikasi terpenuhi	Sebagian besar butir terpenuhi	Hanya sebagian kecil yang berjalan
Tidak memblokir dan tepat waktu	15	Tanpa `delay()` di `loop()` , selisih `millis()` benar	Ada `delay()` pendek yang belum mengganggu	`delay()` panjang membuat tombol terlewat

Aspek	Bobot	Penuh	Sebagian	Minim
Kualitas kode	15	Nama jelas, konstanta pin dan waktu, fungsi pembantu, komentar "mengapa"	Cukup rapi	Sulit dibaca, angka "ajaib" bertebaran
Penjelasan lisan dan modifikasi	15	Bisa menjelaskan setiap bagian dan langsung melakukan perubahan yang diminta	Menjelaskan sebagian	Tidak bisa menjelaskan kodenya sendiri

🚦 Latihan Proyek Mini: Lampu Penyeberangan Pintar

📁 *Folder*: `kode/01-latihan-proyek-mini`

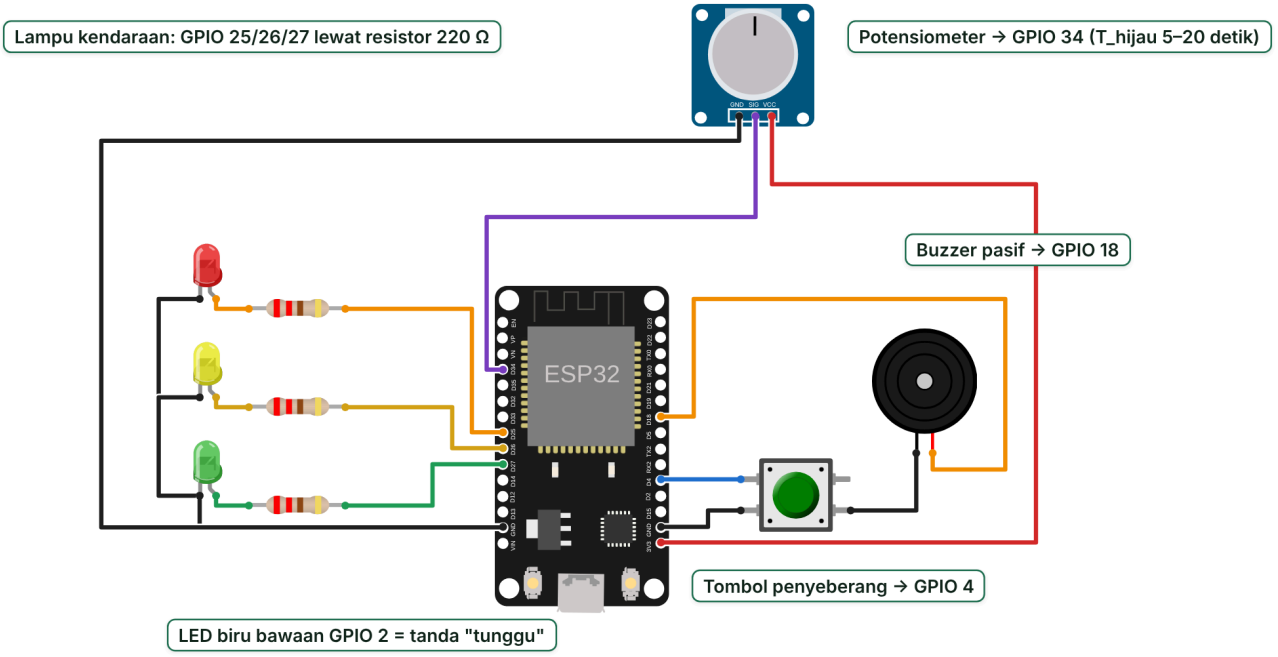
Latihan ini sengaja dibuat sedikit lebih besar daripada soal praktik UTS. Kalau kamu bisa menyelesaikannya dalam ±100 menit, kamu siap. Kerjakan dulu sendiri, baru buka kunci jawaban di akhir bagian ini.

Cerita dan Spesifikasi

Di depan kampus ada penyeberangan jalan. Supaya lalu lintas lancar, lampu kendaraan dibiarkan **hijau** dan baru berubah ketika ada pejalan kaki yang menekan tombol. Namun, kendaraan tetap dijamin mendapat lampu hijau minimal beberapa detik. Tugasmu membuat pengendalinya dengan ESP32.

No	Syarat	Topik
S1	Lampu kendaraan merah, kuning, dan hijau di GPIO 25, 26, dan 27, masing-masing lewat resistor 220 Ω. Saat mulai, hanya hijau yang menyala.	P03
S2	Tombol penyeberang di GPIO 4 (`INPUT_PULLUP`) dibaca dengan **interupsi**. Satu tekanan = satu permintaan; pantulan dan saat tombol dilepas tidak boleh terhitung.	P04, P05
S3	Permintaan hanya dicatat saat lampu hijau. LED biru bawaan (GPIO 2) menyala sebagai tanda "tunggu" sampai penyeberangan dimulai. Tekanan di tahap lain diabaikan dan dilaporkan di Serial.	P03, P05
S4	Hijau dipertahankan minimal T_{hijau} = 5–20 detik, diatur potensiometer GPIO 34 secara langsung. Nilai baru dicetak hanya bila berubah cukup berarti.	P06
S5	Setelah ada permintaan dan T_{hijau} terpenuhi: kuning 3 s → merah 10 s (pejalan menyeberang) → merah 2 s (jeda aman) → hijau.	P03, P05
S6	Selama menyeberang, *buzzer* pasif GPIO 18 berbunyi 1.000 Hz selama 100 ms setiap 500 ms; pada 3 detik terakhir setiap 200 ms.	P07
S7	Tidak ada `delay()` di `loop()`. Semua pewaktuan memakai `millis()`.	P05
S8	Serial Monitor (115200) mencetak T_{hijau} dan setiap pergantian tahap.	P02

Rangkaian



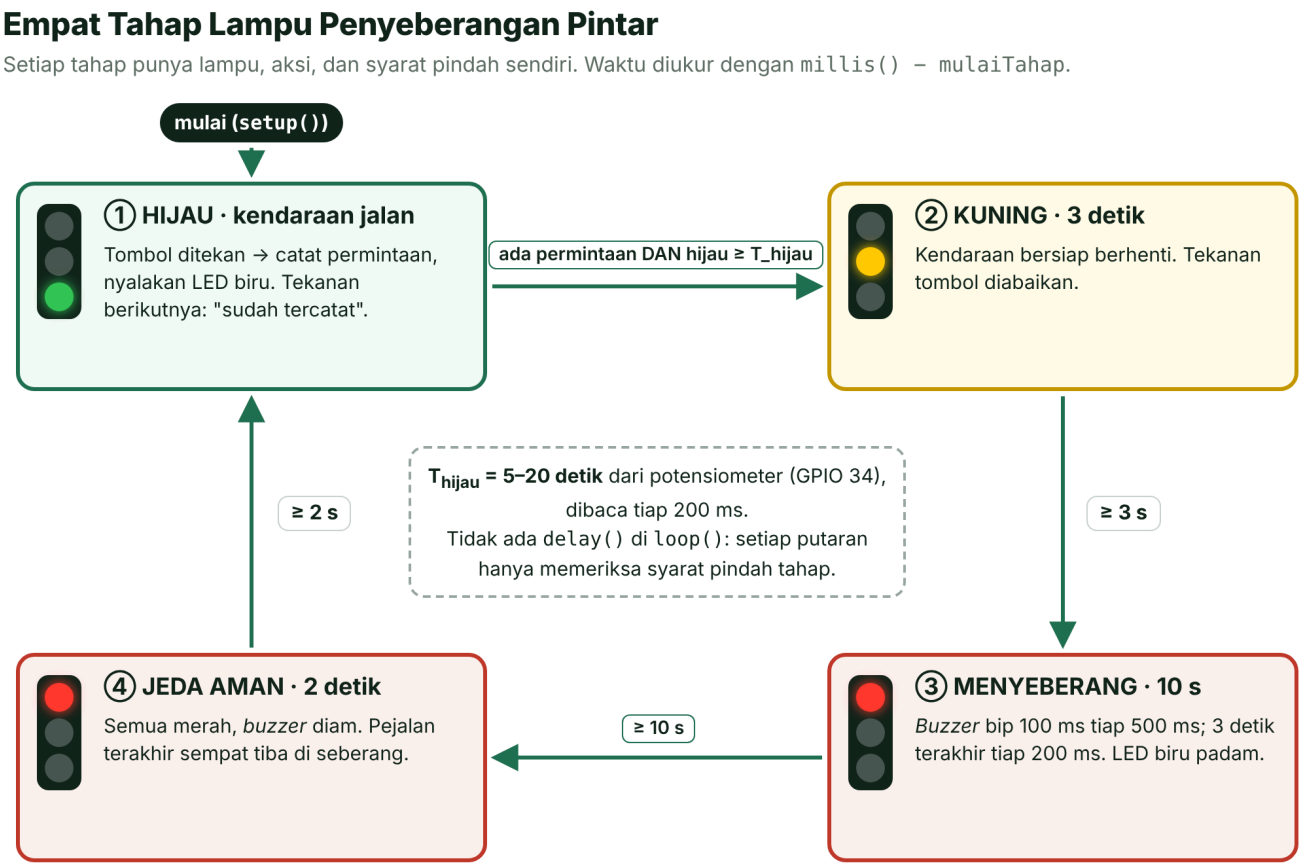
Gambar 8.3 — Rangkaian lampu penyeberangan pintar. Sumber: dibuat untuk materi ini dari `diagram.json`; gambar komponen dari @wokwi/elements (MIT).

Komponen	Kaki komponen	Sambung ke
LED merah	Anoda lewat resistor 220 Ω; katode	GPIO 25; GND
LED kuning	Anoda lewat resistor 220 Ω; katode	GPIO 26; GND
LED hijau	Anoda lewat resistor 220 Ω; katode	GPIO 27; GND
Tombol penyeberang	Dua kaki diagonal	GPIO 4; GND
Buzzer pasif	Kaki + ; kaki –	GPIO 18; GND
Potensiometer 10 kΩ	Kaki tengah (SIG); dua kaki pinggir	GPIO 34; 3V3 dan GND
LED biru bawaan	Sudah terpasang di *board*	GPIO 2

> **Hati-hati / Caution**
>
> **Potensiometer ke 3V3, bukan ke VIN.** Pin VIN bertegangan ±5 V saat *board* dicatu USB. Jika kaki pinggir potensiometer disambung ke VIN, kaki tengahnya bisa memberi lebih dari 3,3 V ke GPIO 34 dan merusak pin.

Tahap dan Pewaktuan

Program ini bekerja dalam empat **tahap**. Setiap tahap punya lampu, aksi, dan syarat pindah sendiri. Di setiap putaran, `loop()` hanya memeriksa: "Sudah waktunya pindah tahap?" Karena tidak ada yang menunggu, tombol dan potensiometer tetap terbaca kapan pun. Di Pertemuan 11, pola ini akan kamu pelajari secara resmi sebagai ***finite state machine*** (FSM, mesin keadaan hingga).

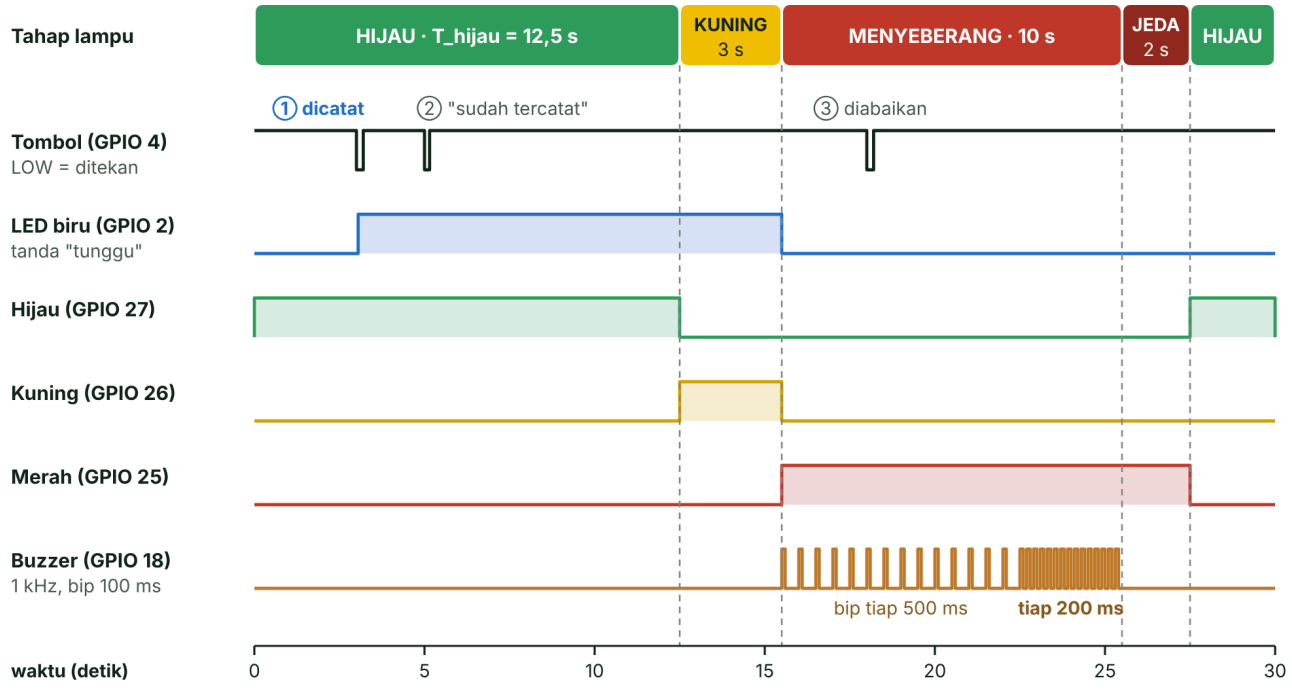


Gambar 8.4 — Empat tahap lampu penyeberangan dan syarat pindahnya. Sumber: dibuat untuk materi ini.

Perhatikan bahwa T_{hijau} dihitung sejak lampu **mulai hijau**, bukan sejak tombol ditekan. Jika pejalan menekan tombol setelah hijau berlangsung lebih lama dari T_{hijau}, lampu langsung berganti kuning. Diagram waktu berikut memperlihatkan satu siklus lengkap.

Diagram Waktu Satu Siklus Penyeberangan

Contoh: potensiometer di tengah (T_{hijau} = 12,5 s), tombol ditekan pada detik ke-3, ke-5, dan ke-18.



Gambar 8.5 — Diagram waktu satu siklus (T_{hijau} = 12,5 s; tombol ditekan pada detik ke-3, ke-5, dan ke-18). Sumber: dibuat untuk materi ini.

Langkah Pengerjaan yang Disarankan

1. **Lampu otomatis dulu.** Buat empat tahap dengan `millis()` dan anggap sementara selalu ada permintaan. Pastikan urutan dan durasinya benar di Serial Monitor.
2. **Tambahkan tombol.** Pasang ISR `FALLING` yang hanya mencatat kejadian, lalu lakukan *debounce* di `loop()`. Catat permintaan dan nyalakan LED biru.
3. **Tambahkan potensiometer.** Baca setiap 200 ms, petakan ke 5.000–20.000 ms, dan cetak hanya bila berubah cukup jauh.
4. **Tambahkan *buzzer*.** Pakai sisa bagi (`%`) dari lama tahap untuk membuat irama bip.
5. **Rapikan.** Ganti angka "ajaib" dengan konstanta, beri komentar "mengapa", dan cek ulang setiap syarat S1–S8.

> **Tips / Tip**
>
> **Simpan setiap langkah yang berhasil.** Salin `main.cpp` ke berkas cadangan sebelum menambah fitur berikutnya. Saat ujian, kebiasaan ini menyelamatkan nilaimu jika fitur baru membuat program kacau menjelang waktu habis.

Cara Menjalankan

Jalur A — Wokwi: buka https://wokwi.com/projects/new/esp32, salin isi `src/main.cpp` ke tab **sketch.ino** dan isi `diagram.json` ke tab **diagram.json**, lalu klik ▶. Klik tombol hijau untuk menyeberang dan geser kenop potensiometer untuk mengubah T_{hijau}.

Jalur B — *board* asli: rangkai sesuai Gambar 8.3, buka *folder* `pertemuan-08-uts/kode/01-latihan-proyek-mini` di VS Code, klik **Build**, **Upload**, lalu **Serial Monitor**.

Hasil yang Diharapkan

Skenario: kenop potensiometer di tengah, tombol ditekan sekitar detik ke-3, ditekan lagi sebelum lampu kuning, dan ditekan sekali saat penyeberangan berlangsung.

```
[INFO] Lampu penyeberangan pintar siap.
[INFO] Durasi hijau minimum: 12.5 detik
[TAHAP] HIJAU       : kendaraan jalan; tekan tombol untuk menyeberang
[TOMBOL] Permintaan menyeberang tercatat.
[TOMBOL] Permintaan sudah tercatat, harap tunggu.
[TAHAP] KUNING      : kendaraan bersiap berhenti
[TAHAP] MENYEBERANG : kendaraan berhenti, pejalan kaki menyeberang
[TOMBOL] Diabaikan: siklus penyeberangan sedang berjalan.
[INFO] Sisa 3 detik: bip dipercepat
[TAHAP] JEDA AMAN   : semua merah, penyeberangan dikosongkan
[TAHAP] HIJAU       : kendaraan jalan; tekan tombol untuk menyeberang
```

Angka pada baris kedua bergantung pada posisi kenop (5,0–20,0 detik), dan baris baru `[INFO] Durasi hijau minimum: …` muncul setiap kali kenop diputar cukup jauh. Selama tahap MENYEBERANG, *buzzer* berbunyi 14 kali dengan irama lambat lalu 15 kali dengan irama cepat.

Kalau Belum Berhasil

Gejala	Coba ini
Satu tekanan tercatat dua kali ("sudah tercatat" muncul sendiri)	Pantulan atau tepi saat tombol dilepas ikut terhitung. Pastikan tekanan baru diterima setelah tenang 50 ms **dan** tombol masih `LOW`.
Tombol tidak bereaksi sama sekali	Cek `INPUT_PULLUP`, mode `FALLING`, dan kaki tombol yang dipakai harus diagonal (tidak sekelompok).
T_{hijau} tidak berubah saat kenop diputar	Pastikan kaki tengah potensiometer ke GPIO 34 dan kedua kaki pinggir ke 3V3 dan GND.
Serial Monitor dibanjiri nilai potensiometer	Cetak hanya bila nilai ADC berubah ≥ 40 (±1%) dari nilai terakhir yang dicetak.
Buzzer diam	Pastikan *buzzer* **pasif** dan `ledcAttach()` mengembalikan `true`. *Buzzer* aktif hanya berbunyi dengan satu nada bawaannya.
Lampu tidak pernah kuning	Periksa syarat pindah: permintaan harus `true` **dan** `millis() - mulaiTahap >= durasiHijauMs`.

Rubrik Latihan Proyek Mini (100 Poin)

Aspek	Poin	Kriteria penuh
Rangkaian	15	Tiga LED dengan resistor, tombol ke GND dengan `INPUT_PULLUP`, potensiometer ke 3V3, *buzzer* pasif

Aspek	Poin	Kriteria penuh
Urutan tahap dan durasi	20	Hijau → kuning 3 s → merah 10 s → jeda 2 s → hijau, sesuai syarat S1 dan S5
Tombol	15	Interupsi + *debounce*; satu tekanan = satu permintaan; hanya dicatat saat hijau; LED biru benar
Potensiometer	10	T_{hijau} 5–20 s langsung berpengaruh; Serial tidak banjir
Buzzer berirama	15	Dua irama sesuai S6, berhenti tepat di akhir penyeberangan
Tidak memblokir	10	Tanpa `delay()` di `loop()`; semua pewaktuan dengan selisih `millis()`
Kualitas kode dan Serial	10	Konstanta, fungsi pembantu, komentar "mengapa", pesan Serial jelas
Penjelasan lisan	5	Bisa menjelaskan alur tahap, ISR, dan *debounce*

Kunci Jawaban

Kode lengkap ada di `kode/01-latihan-proyek-mini/src/main.cpp` dan sudah lulus kompilasi dengan Arduino-ESP32 3.3. Buka hanya setelah kamu mencoba sendiri.

▶ Lihat kode solusi lengkap

Pengembangan (Opsional)

- Tambahkan lampu pejalan kaki merah dan hijau di GPIO 32 dan 33 (masing-masing lewat 220 Ω) yang menyala berlawanan dengan lampu kendaraan.
- Cetak hitung mundur sisa waktu menyeberang setiap detik di Serial Monitor.
- Tekan-tahan tombol ≥ 2 detik untuk memperpanjang waktu menyeberang 50%, misalnya bagi lansia atau penyandang disabilitas.

💡 Tips Belajar

- **Buat lembar ringkasmu sendiri.** Menulis ulang rumus dan API dengan tangan membuatmu lebih ingat, walaupun saat ujian lembar ringkas resmi disediakan.
- **Ulangi praktikum P03–P07 tanpa melihat kode.** Rangkai di Wokwi, tulis dari nol, lalu bandingkan dengan modul. Bagian yang terlupa adalah bahan belajarmu.
- **Latih hitungan dengan kalkulator.** Resistor LED, konversi ADC, pembagi tegangan, siklus kerja, batas LEDC, nilai alarm *timer*, dan selisih `millis()` hampir pasti muncul.
- **Kerjakan ulang "Cek Pemahaman" dan "Latihan & Tantangan"** di setiap modul. Untuk tiap jawaban, tanyakan "mengapa", bukan hanya "apa".
- **Simulasikan ujian praktik.** Kerjakan latihan proyek mini di atas dengan pengatur waktu 100 menit, tanpa membuka kunci jawaban.
- **Biasakan membaca galat kompilasi.** Baca baris pertama galat, buka nomor barisnya, dan periksa juga baris di atasnya.
- **Siapkan alat sehari sebelumnya.** Pastikan Wokwi bisa dibuka, PlatformIO bisa **Build**, serta kabel USB data dan *driver* berfungsi bila memakai *board*.

📏 Tata Tertib Ujian

Tata tertib berikut adalah usulan *(sesuaikan)*.

1. Hadir paling lambat 15 menit sebelum ujian dan membawa kartu identitas mahasiswa.
2. Ponsel dimatikan dan disimpan di tas selama ujian teori. Kalkulator biasa boleh dipakai, kalkulator di ponsel tidak.
3. Ujian teori bersifat buku tertutup; hanya lembar ringkas resmi dari pengawas yang boleh ada di meja.
4. Saat ujian praktik, kamu boleh membuka modul P01–P07 dan dokumentasi resmi. Memakai AI, aplikasi pesan, berbagi berkas, atau menyalin kode teman **dilarang**.
5. Simpan pekerjaan secara berkala dan kumpulkan dengan nama berkas sesuai ketentuan, misalnya `UTS_NIM_Nama.zip`.
6. Cabut USB sebelum mengubah rangkaian. Laporkan kepada pengawas bila ada komponen panas, berbau, atau *board* rusak; kamu akan diminta melanjutkan di Wokwi.
7. Pelanggaran integritas akademik ditangani sesuai aturan program studi. Ingat, setiap kode yang kamu kumpulkan harus bisa kamu jelaskan sendiri.

🚫 Kesalahan Umum

Kesalahan	Akibat	Cara menghindari
LED tanpa resistor atau terpasang terbalik	LED tidak menyala, pin bisa rusak	Hitung R = (3,3 − V_f) ÷ I; kaki panjang = anoda
Tombol dengan `INPUT` biasa tanpa resistor	Bacaan acak (mengambang)	Pakai `INPUT_PULLUP`; ingat ditekan = `LOW`
Memakai GPIO 34–39 sebagai output atau dengan `INPUT_PULLUP`	Tidak berfungsi	Pin itu input saja tanpa *pull-up* internal
Sensor analog di ADC2 (mis. GPIO 4, 25–27) bersama Wi-Fi	Bacaan gagal	Pakai ADC1: GPIO 32–39
`delay()` panjang di `loop()`	Tombol terlewat, program terasa "beku"	Pola selisih `millis()`
`if (millis() >= waktuBerikutnya)`	Salah di sekitar luapan	`if (sekarang - sebelumnya >= INTERVAL)`
Menyimpan `millis()` di `int`	Bernilai negatif setelah ±24,9 hari	Pakai `unsigned long`
`Serial` atau `delay()` di ISR, variabel bersama tanpa `volatile`	*Crash* atau nilai basi	ISR singkat + `IRAM_ATTR` + `volatile` + bagian kritis
Mode `RISING` untuk tombol dengan *pull-up*	Aksi terjadi saat tombol dilepas	Pakai `FALLING`
`nilai / 4095 * 3.3` dengan `nilai` bertipe `int`	Selalu 0 karena pembagian bulat	`nilai * 3.3 / 4095` atau `nilai / 4095.0 * 3.3`
Memakai `ledcSetup()` / `ledcAttachPin()`	Galat kompilasi di core 3.x	`ledcAttach(pin, frek, bit)`

Kesalahan	Akibat	Cara menghindari
Frekuensi × 2^n > 80 MHz	`ledcAttach()` gagal, PWM tidak keluar	Turunkan resolusi atau frekuensi
Potensiometer atau sensor dicatu 5 V lalu ke pin	ADC jenuh, pin bisa rusak	Catu dari 3V3 atau pakai pembagi tegangan
Beban kumparan tanpa dioda *flyback*	Transistor rusak	Dioda paralel kumparan, katode ke +5 V

📚 Referensi

1. Espressif Systems. (2025). *ESP32 Series Datasheet* (v5.x). https://documentation.espressif.com/esp32_datasheet_en.pdf
2. Espressif Systems. (2026). *ESP32-WROOM-32E & ESP32-WROOM-32UE Datasheet*. https://documentation.espressif.com/esp32-wroom-32e_esp32-wroom-32ue_datasheet_en.pdf
3. Espressif Systems. (2026). *Arduino-ESP32 Documentation: GPIO, ADC, LEDC, dan Timer API*. https://docs.espressif.com/projects/arduino-esp32/en/latest/
4. CodeMagic LTD. (2026). *Wokwi Docs*. https://docs.wokwi.com
5. Lee, E. A., & Seshia, S. A. (2017). *Introduction to Embedded Systems: A Cyber-Physical Systems Approach* (2nd ed.). MIT Press. https://ptolemy.berkeley.edu/books/leeseshia/
6. Modul Pertemuan 01–07 mata kuliah ini, terutama bagian "Rangkuman" setiap modul.

🖼️ Sumber Gambar

Gambar	Sumber dan lisensi
8.1, 8.4, 8.5	Dibuat untuk materi ini.
8.2	Dibuat untuk materi ini. Ikon: Font Awesome Free (Fonticons, CC BY 4.0).
8.3	Dibuat untuk materi ini dari `diagram.json`; gambar komponen dari @wokwi/elements (MIT, © 2020 Uri Shaked).

⬅️ Pertemuan 07: PWM dan Mengendalikan Beban · 🏠 Beranda · Pertemuan 09: Komunikasi Serial ➡️